Simon Sutoris 7542170

Evaluation Contexts

The upper rules are computation rules, the lower congruence rules.

Computation rules:

$$\begin{array}{rcr} v := & n & Values \\ v_1 \oplus v_2 \to & & (v_1 \oplus v_2) \\ \texttt{ifz}(0; t_1, t_2) \to & & t_1 \\ \texttt{ifz}(v; t_2, t_2) \to & & t_2 \text{ if } n \neq 0 \end{array}$$

Congruence rules:

$$\frac{t_1 \to t_1'}{t_1 \oplus t_2 \to t_1' \oplus t_2} \quad \frac{t_2 \to t_2'}{v \oplus t_2 \to v \oplus t_2'} \quad \frac{t_1 \to t_1'}{\texttt{ifz}(t_1; t_2, t_3) \to \texttt{ifz}(t_1'; t_2, t_3)}$$

Congruence rules re-formulated using *evaluation contexts*:

$$\begin{array}{rl} E ::= & \\ & [\,] \\ & E \oplus t \\ & v \oplus E \\ & \texttt{ifz}(E; t_2, t_3) \end{array}$$

Evaluation

Derivation Trees

$$\frac{\dfrac{3 * 4 \to 12}{(3 * 4) - 5 \to 12 - 5}}{\texttt{ifz}((3 * 4) - 5;\ (7 * 7) + 15,\ 4 + 4)\ 7 \to \texttt{ifz}(12 - 5;\ (7 * 7) + 15,\ 4 + 4)}$$

Normal Forms

A term which is in normal form and is a value is quite simple. Any term in the form of $t = v$ solves the task, e.g: $t_1 = 42$. However, there exists no term which is in normal form and is not a value.

Lemma 1. *Every term t, which is not a value, is not in normal form.*

Proof. By contraposition: Every term t, which is in normal form, is a value.
Due to it being in normal form, no congruence rules and almost no computation rules apply. The only rule left is $v := n$. □

Lemma 2. *Every term t, which is not in normal form, is not a value.*

Proof. The grammer of the language only allows for three types of terms: values; concatenation between terms using binary operations; or using `ifz`, which consumes 3, possibly different terms.

To not be in normal form, a term has to be found which cannot be evaluated. A value would fit this requirement, but the question also asks for a term that is not a value. Therefore, a concatenation between binary operations or usage of `ifz` has to be found.

Let's first look at the binary operations. Three cases:

(i) $t_1 = v_1, t_2 = v_2$:

Computation rule 2 applies.

(ii) $t_1 = v_1, t_2 = t_2$:

t_2 is either concatenated with a binary operator, or uses `ifz`, both of which can be evaluated. Congruence rule 2 applies.

(iii) $t_1 = t_1, t_2 = v_2 \vee t_1 = t_1, t_2 = t_2$:

Analog to (ii), t_1 is either concatenated with a binary operator, or uses `ifz`. Congruence rule 1 applies.

Secondly, let's look at `ifz`. Three cases:

(iv) $n = 0$:

Computation rule 3 applies.

(v) $n! = 0$:

Computation rule 4 applies.

(vi) $n! = t$ for $t \to t'$:

Congruence rule 3 applies.

Of course, (ii), (iii) and (vi) are recursive. However, the `size` of a term never grows. It only shrinks, or stays the same when $t = v$, as described in Section . Therefore, for each term, a computation step exists, meaning there exists no normal form for a term that is not a value.

□

Because Lemma 1 and 2 hold, there cannot exists a term, which is in normal form and is not a value. ■

To create a term that is stuck, we would need to build an infinite term, like $t = t_1 \oplus t_2 \oplus ...$, or introduce a syntax error, for example $t = foo$, $t = +4\ 3$, or $t = \texttt{ifz}(1; 2)$, but the grammar of our language prohibits terms like these, which means these terms are not part of the language.

Induction on Derivation Trees

Lemma 3. $\mathit{size}(t) \geq \mathit{size}(t') \mid t \to t'$

Proof. Three cases.

(i) $t = v$

$$\texttt{size}(t) = \texttt{size}(v) = 1 = \texttt{size(t')} \tag{1}$$

(ii) $t = t_1 \oplus t_2 \mid t_1 \to^* v_1$

$$\begin{aligned}
\texttt{size}(t_1 \oplus t_2) &= \texttt{size}(t_1) + \texttt{size}(t_2) + 1 && (2)\\
&\geq \texttt{size}(v_1) + \texttt{size}(t_2) + 1 && (3)\\
&= \texttt{size}(t') && (4)\\
\Leftrightarrow \texttt{size}(t_1) &\geq \texttt{size}(v_1) && (5)\\
\texttt{size}(t_1) &\geq 1 && (6)
\end{aligned}$$

Analog for $t = t_1 \oplus t_2, t_2 \to^* v_2$

(iii) $t = \texttt{ifz}(t_1; t_2, t_3) \mid t_1 \to^* v_1$

$$\begin{aligned}
\texttt{size}(\texttt{ifz}(t_1; t_2, t_3)) &= \texttt{size}(t_1) + \texttt{size}(t_2) + \texttt{size}(t_3) + 1 && (7)\\
&\geq \texttt{size}(v_1) + \texttt{size}(t_2) + \texttt{size}(t_3) + 1 && (8)\\
&= \texttt{size}(t') && (9)\\
\Leftrightarrow \texttt{size}(t_1) &\geq \texttt{size}(v_1) && (10)\\
\texttt{size}(t_1) &\geq 1 && (11)
\end{aligned}$$

□

Types and Programming Languages

Is OCaml mandatory or can we use it at home to verify the contents of the book? Also, are the questions mandatory? Chapter 4 was rather short compared to the beginnings of the book till Chapter 3, and I feel like I have to force myself to come up with a question?